

Design pattern per esempi

I design pattern strutturali

I design pattern di tipo strutturale riguardano le modalità con cui classi e oggetti vengono aggregati allo scopo di formare entità più complesse. In generale possiamo distinguere due tipologie di pattern strutturali:

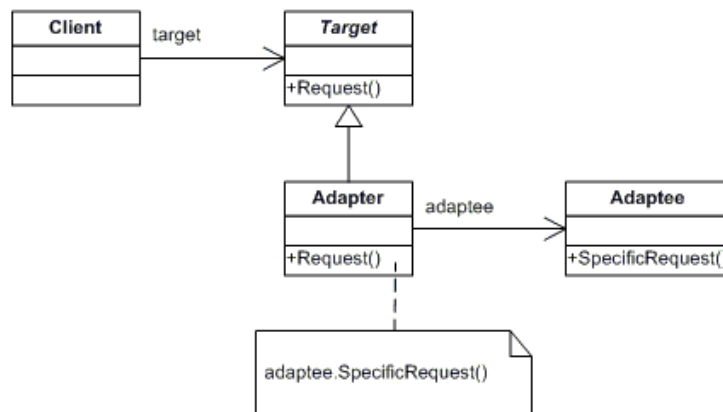
- pattern strutturali basati sulle classi, che sfruttano l'ereditarietà multipla (se prevista) per combinare tra loro le interfacce e le implementazioni;
- pattern strutturali basati sugli oggetti, che descrivono le modalità di composizione di oggetti al fine di estendere in fase di esecuzione le funzionalità di una classe (cosa non possibile nel caso della composizione statica e tramite ereditarietà).

In generale la maggior parte dei pattern strutturali sono basati sugli oggetti. Il pattern Adapter è un esempio di pattern basato sia sulle classi che sugli oggetti.

Adapter

L'Adapter (detto anche Wrapper) è un pattern strutturale basato sulle classi e sugli oggetti che ha lo scopo di convertire l'interfaccia di una classe in un'altra interfaccia. Questo pattern permette a classi diverse di operare insieme anche nel caso in cui questo non sarebbe possibile per via delle interfacce tra loro non compatibili.

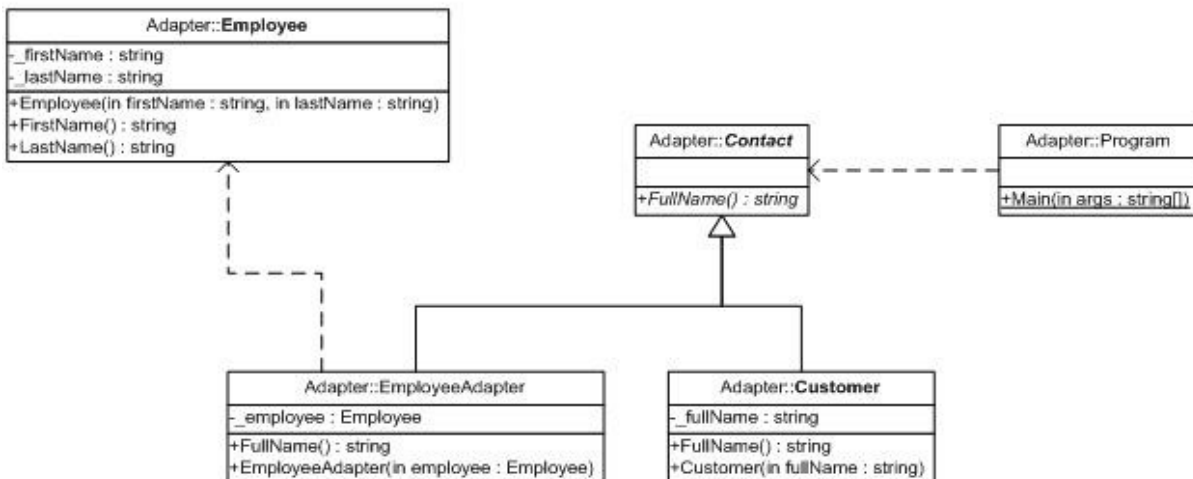
Questo pattern trova una forte applicabilità soprattutto in quegli scenari dove oggetti simili appartenenti ad ambienti o sistemi diversi hanno la necessità di interoperare tra loro. In questi casi l'utilizzo di un oggetto wrapper consente di adattare tra loro oggetti strutturalmente organizzati in modo diverso, di farli comunicare tra loro e di metterli in correlazione.



I partecipanti di questo pattern sono (tra parentesi sono indicati gli oggetti equivalenti nell'esempio proposto successivamente):

- **Target** (Contact)
Definisce l'interfaccia di riferimento alla quale l'oggetto *Adaptee* si deve adattare.
- **Adaptee** (Employee)
Rappresenta l'interfaccia che deve essere adattata.
- **Adapter** (EmployeeAdapter)
Adatta l'interfaccia di *Adaptee* all'interfaccia di *Target*.
- **Client** (Program)
Utilizza unicamente oggetti compatibili con l'interfaccia di *Target*.

L'esempio proposto per questo pattern include due classi Employee e Customer con interfacce diverse. La classe Employee include due proprietà FirstName e LastName che rappresentano il nome e il cognome dell'impiegato. La classe Customer (cliente) deriva direttamente dalla classe base astratta Contact che include un'unica proprietà FullName che rappresenta il nome completo del contatto, composto dalla concatenazione di nome e cognome. In questo scenario non è possibile assegnare direttamente un'istanza di Employee a un riferimento di tipo Contact. Per poterlo fare, occorre adattare l'interfaccia di Employee a quella di Contact. La classe EmployeeAdapter deriva da Contact e presenta un campo privato di tipo Employee. In fase di creazione l'istanza della classe Employee da adattare viene passata all'adapter come parametro sul costruttore. La proprietà FullName di EmployeeAdapter esegue la concatenazione del nome e del cognome dell'impiegato utilizzando le proprietà dell'oggetto interno.



```

using System;
using System.Collections.Generic;
using System.Text;

namespace DesignPatterns.Adapter
{
    public class Employee
    {
        private string _firstName;
        private string _lastName;

        public Employee(string firstName, string lastName)
        {
            _firstName = firstName; _lastName = lastName;
        }

        public string FirstName
        {
            get { return _firstName; }
        }

        public string LastName
        {
            get { return _lastName; }
        }
    }

    public abstract class Contact
    {
        public abstract string FullName { get; }
    }
}

```

```

public class Customer : Contact
{
    private string _fullName;

    public Customer(string fullName)
    {
        _fullName = fullName;
    }

    public override string FullName
    {
        get { return _fullName; }
    }
}

public class EmployeeAdapter : Contact
{
    private Employee _employee;

    public EmployeeAdapter(Employee employee)
    {
        _employee = employee;
    }

    public override string FullName
    {
        get { return _employee.FirstName + " " + _employee.LastName; }
    }
}

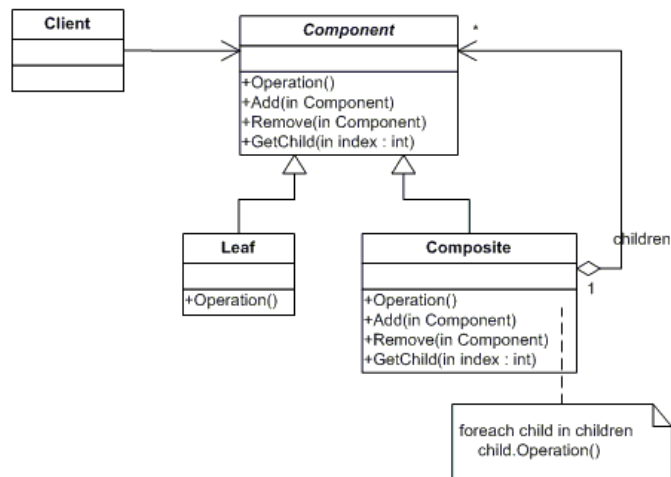
public class Program
{
    public static void Main(string[] args)
    {
        Contact c = new Customer("Marco Rossi");
        Console.WriteLine(c.FullName);
        c = new EmployeeAdapter(new Employee("Marco", "Rossi"));
        Console.WriteLine(c.FullName);
    }
}

```

Composite

Il pattern Composite, di tipo strutturale basato sugli oggetti, consente di creare gerarchie di oggetti aggregando insieme elementi primitivi e composti direttamente a runtime. Questo pattern può essere applicato per rappresentare strutture ad albero in una logica parte-tutto (*part-whole*), dove ciascun elemento può a sua volta aggregare insieme altri elementi della stessa specie e dove oggetti singoli e composizioni possono essere trattati in modo uniforme.

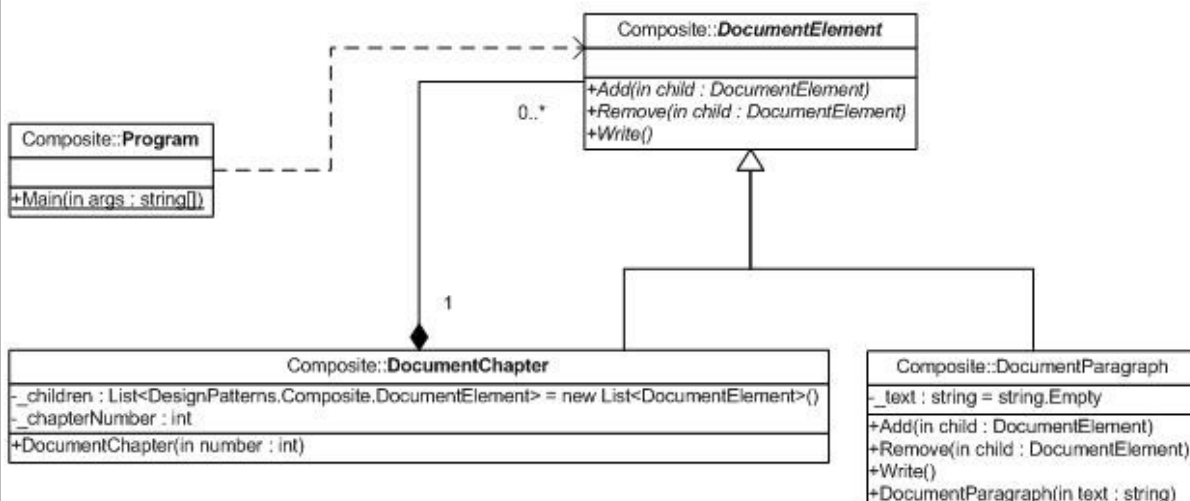
Uno dei pregi principali nell'applicazione di questo pattern risiede nel fatto di rendere molto agevole l'aggiunta di nuove tipologie di oggetti componenti. In più, dal momento che i vari elementi vengono trattati in modo uniforme, l'utilizzo della struttura composita nell'ambito del client risulta essere molto semplificata. Ciascun elemento, seppur presentando caratteristiche simili, può essere caratterizzato da comportamenti assai diversi, a seconda dei casi. Il client non conosce la differenza che esiste tra i diversi elementi e, in particolare, tra oggetti primitivi e oggetti composti, pertanto tratta entrambe le tipologie allo stesso modo, con un significativo miglioramento della leggibilità del codice.



I partecipanti di questo pattern sono (tra parentesi sono indicati gli oggetti equivalenti nell'esempio proposto successivamente):

- **Component** (DocumentElement)
Fornisce l'interfaccia di riferimento valida per tutti gli elementi della struttura ad albero, ovvero sia per gli elementi terminali che per gli elementi intermedi. Rappresenta la classe base per *Composite* (oggetto composto) e per *Leaf* (oggetto primitivo).
- **Composite** (DocumentChapter)
Definisce il comportamento degli elementi intermedi che hanno figli e che aggregano insieme altri *Component*.
- **Leaf** (DocumentParagraph)
Definisce il comportamento degli elementi terminali che non hanno figli e che rappresentano gli oggetti primitivi.
- **Client** (Program)
Utilizza la struttura composta, accedendo ai vari elementi tramite l'interfaccia *Component*.

Consideriamo l'organizzazione di un documento suddiviso in capitoli e paragrafi rappresentati da altrettanti oggetti il cui tipo base è la classe astratta *DocumentElement*. Sebbene la struttura del documento considerata è volutamente molto semplificata per non introdurre una complessità tale da limitare l'efficacia e la chiarezza dell'esempio, il documento risulta essere un'aggregazione più o meno complessa dei suoi elementi costituenti. Nell'esempio ciascun capitolo (classe *DocumentChapter*) è un elemento composto che contiene uno o più paragrafi. L'elemento terminale della struttura del documento è rappresentato dal paragrafo (classe *DocumentParagraph*).



Occorre fare alcune considerazioni. Si noti innanzitutto che, a parità di interfaccia, i diversi oggetti di tipo `DocumentElement` mostrano comportamenti peculiari a seconda dei casi: l'implementazione interna dell'elemento terminale (il paragrafo) è differente da quella dell'elemento composito (il capitolo). Questo fa sì che, nella rappresentazione del documento, il metodo `Write(string)` possa essere eseguito in modo iterativo mantenendo il codice molto essenziale e senza la necessità di introdurre strutture particolari di controllo del flusso, dal momento che ogni singola istanza presenta un comportamento particolare in funzione del tipo di appartenenza. Si noti inoltre che nulla vieta che un capitolo possa a sua volta contenere uno o più sottocapitoli sempre di tipo `DocumentChapter`, dato che gli elementi della lista dei suoi figli sono in ogni caso di tipo `DocumentElement` (il tipo base). Si noti infine come l'aggiunta di una nuova tipologia di elemento (per esempio, una ipotetica classe `DocumentSection`) non rappresenti un grosso problema, dal momento che la struttura è pensata per evolvere in modo flessibile nel tempo.

```
using System;
using System.Collections.Generic;

namespace DesignPatterns.Composite
{
    public abstract class DocumentElement
    {
        public abstract void Add(DocumentElement child);
        public abstract void Remove(DocumentElement child);
        public abstract void Write();
    }

    public class DocumentChapter : DocumentElement
    {
        private int _chapterNumber;
        private List<DocumentElement> _children = new List<DocumentElement>();

        public DocumentChapter(int number)
        {
            _chapterNumber = number;
        }

        public override void Add(DocumentElement child)
        {
            _children.Add(child);
        }

        public override void Remove(DocumentElement child)
        {
            _children.Remove(child);
        }

        public override void Write()
        {
            Console.WriteLine("Chapter " + _chapterNumber.ToString());
            foreach (DocumentElement child in _children)
                child.Write();
        }
    }

    public class DocumentParagraph : DocumentElement
    {
        private string _text = string.Empty;
        public DocumentParagraph(string text) { _text = text; }

        public override void Add(DocumentElement child)
        {
            throw new NotSupportedException();
        }
    }
}
```

```

public override void Remove(DocumentElement child)
{
    throw new NotSupportedException();
}

public override void Write()
{
    Console.WriteLine(_text);
}
}

public class Program
{
    public static void Main(string[] args)
    {
        DocumentParagraph pg1 = new DocumentParagraph("1.1");
        DocumentParagraph pg2 = new DocumentParagraph("1.2");
        DocumentParagraph pg3 = new DocumentParagraph("2.1");
        DocumentParagraph pg4 = new DocumentParagraph("2.2");

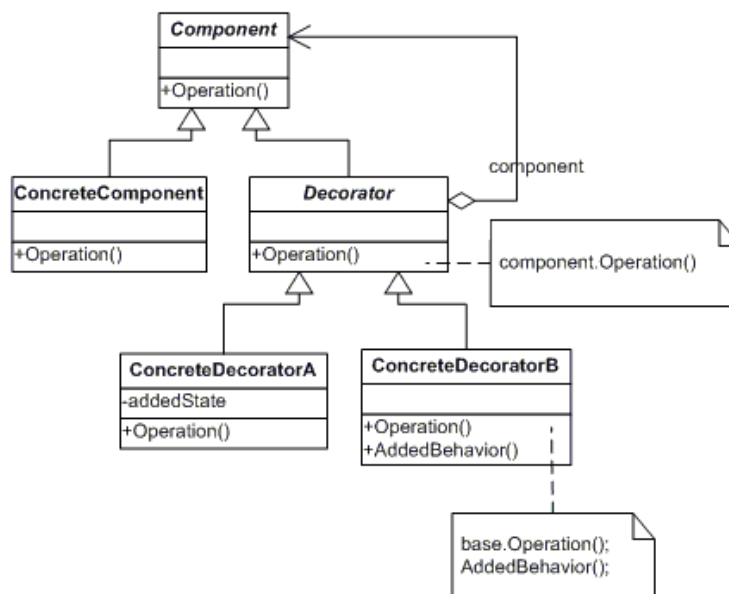
        DocumentChapter chp1 = new DocumentChapter(1);
        chp1.Add(pg1);
        chp1.Add(pg2);
        chp1.Write();

        DocumentChapter chp2 = new DocumentChapter(2);
        chp2.Add(pg3);
        chp2.Add(pg4);
        chp2.Write();
    }
}

```

Decorator

Il design pattern decorator consente di aggiungere metodi a classi esistenti durante il run-time, permettendo una maggior flessibilità nell'aggiungere delle funzionalità agli oggetti. Questo viene realizzato costruendo un nuovo decoratore intorno all'oggetto originale. Normalmente ciò viene realizzato passando l'oggetto originale come parametro al costruttore del decoratore.

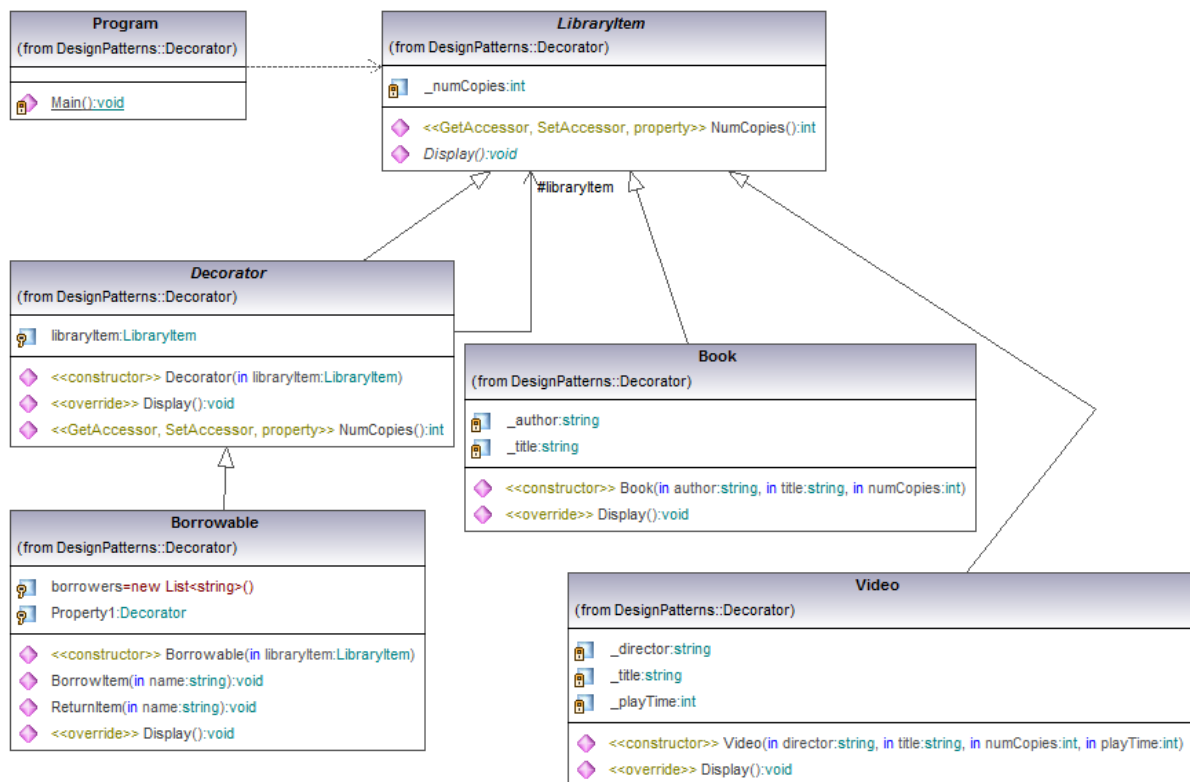


I partecipanti di questo pattern sono (tra parentesi sono indicati gli oggetti equivalenti nell'esempio proposto successivamente):

- **Component** (LibraryItem)
 - Definisce l'interfaccia che aggiunge i metodi dinamicamente. In particolare definisce l'interfaccia per gli oggetti a cui è possibile aggiungere funzionalità dinamicamente.
- **ConcreteComponent** (Book, Video)
 - definisce l'oggetto concreto al quale aggiungere i metodi.
- **Decorator** (Decorator)
 - Mantiene un riferimento all'oggetto Component e definisce un'interfaccia conforme all'interfaccia Component
- **ConcreteDecorator** (Borrowable)
 - Aggiunge i metodi al component.

Il Decorator manda le richieste al Component che può svolgere le operazioni precedenti e successive alla spedizione della richiesta. In questo modo si ottiene una maggior flessibilità, tanti piccoli oggetti al posto di uno molto complicato, andando a modificare il contorno e non la sostanza di una classe.

Il codice mostra un pattern Decorator il quale dinamicamente aggiunge nuove funzionalità ad un oggetto esistente.



```
using System;
using System.Collections.Generic;

namespace DesignPatterns.Decorator
{
    abstract class LibraryItem
    {
        private int _numCopies;
```

```

    public int NumCopies
    {
        get { return _numCopies; }
        set { _numCopies = value; }
    }

    public abstract void Display();
}

class Book : LibraryItem
{
    private string _author;
    private string _title;

    public Book(string author, string title, int numCopies)
    {
        this._author = author;
        this._title = title;
        this.NumCopies = numCopies;
    }

    public override void Display()
    {
        Console.WriteLine("Book {0},{1},{2}", _author, _title, NumCopies);
    }
}

class Video : LibraryItem
{
    private string _director;
    private string _title;
    private int _playTime;

    public Video(string director, string title, int numCopies, int playTime)
    {
        this._director = director;
        this._title = title;
        this.NumCopies = numCopies;
        this._playTime = playTime;
    }

    public override void Display()
    {
        Console.WriteLine("Video {0},{1},{2},{3}", _director, _title,
                          NumCopies, _playTime);
    }
}

abstract class Decorator : LibraryItem
{
    protected LibraryItem libraryItem;

    public Decorator(LibraryItem libraryItem)
    {
        this.libraryItem = libraryItem;
    }

    public override void Display()
    {
        libraryItem.Display();
    }
}

```

```

class Borrowable : Decorator
{
    protected List<string> borrowers = new List<string>();

    public Borrowable(LibraryItem libraryItem)
        : base(libraryItem)
    {
    }

    public void BorrowItem(string name)
    {
        borrowers.Add(name);
        libraryItem.NumCopies--;
    }

    public void ReturnItem(string name)
    {
        borrowers.Remove(name);
        libraryItem.NumCopies++;
    }

    public override void Display()
    {
        base.Display();

        foreach (string borrower in borrowers)
        {
            Console.WriteLine(" borrower: " + borrower);
        }
    }
}

public class Program
{
    static void Main()
    {
        Book book = new Book("Worley", "Inside ASP.NET", 10);
        book.Display();

        Video video = new Video("Spielberg", "Jaws", 23, 92);
        video.Display();

        Console.WriteLine("\nMaking video borrowable:");

        Borrowable borrowvideo = new Borrowable(video);
        borrowvideo.BorrowItem("Customer #1");
        borrowvideo.BorrowItem("Customer #2");

        borrowvideo.Display();
    }
}

```

I design pattern comportamentali

I design pattern comportamentali si riferiscono alla distribuzione delle responsabilità tra oggetti tra loro correlati. Essi non affrontano unicamente gli aspetti relativi alla struttura degli oggetti e delle classi interagenti, ma si focalizzano soprattutto sulle modalità di comunicazione e collaborazione.

La maggior parte di questi pattern fornisce soluzioni per incapsulare le diverse funzionalità in un oggetto specifico con l'intento di delegare ad esso l'esecuzione del codice vero e proprio. Questo approccio permette

in generale di eliminare le dipendenze dirette tra i vari oggetti coinvolti, limitando l'accoppiamento e facilitando la possibilità di estendere e modificare il codice senza grossi sforzi. La distribuzione delle responsabilità porta inevitabilmente a ridurre la genericità di ciascun partecipante dei pattern, aspetto in gran parte dovuto alla forte specializzazione che caratterizza le diverse classi che di volta in volta sono delegate a fornire le funzionalità richieste.

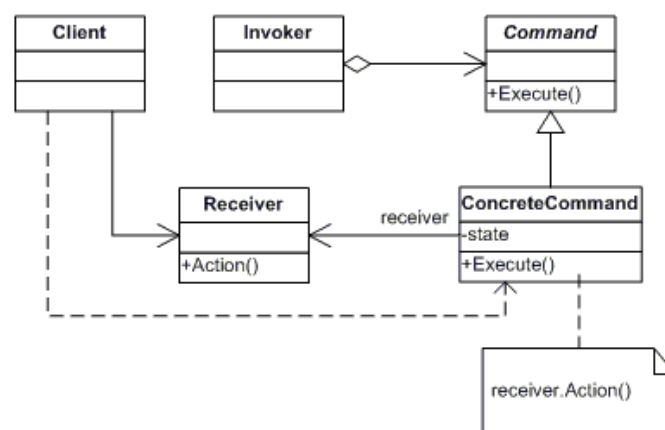
Command

Il pattern Command (noto anche come Action o Transaction) permette di inoltrare richieste ad oggetti senza conoscere assolutamente nulla dell'operazione da eseguire o del destinatario della richiesta. Questo è possibile per il fatto che il pattern in questione tratta la richiesta come un oggetto differente rispetto sia al richiedente che all'oggetto destinatario. Questo oggetto specifica l'azione da svolgere sul destinatario, sfruttandone i comportamenti in modo da tale da poter portare a termine la richiesta.

Il pattern Command permette quindi di incapsulare una richiesta in un oggetto permettendo al client di inoltrare richieste di varia natura, anche in funzione di destinatari diversi. Il pattern in questione può essere applicato:

- per parametrizzare gli oggetti rispetto ad una azione da compiere;
- per specificare, accodare ed eseguire svariate richieste in tempi diversi, anche trasferendo un comando da un contesto di esecuzione ad un altro;
- per consentire l'annullamento delle operazioni eseguite (undo, rollback), mantenendo preventivamente lo stato per annullare gli effetti dei comandi stessi.

Il vantaggio più significativo nell'applicazione di questo pattern è il fatto di ottenere un perfetto disaccoppiamento tra l'oggetto che invoca il comando e il destinatario, ovvero quello che conosce il modo per portare a termine l'operazione. Questo aspetto consente di poter aggiungere comandi ulteriori associati a destinatari diversi in modo abbastanza immediato senza che sia necessario modificare le classi già esistenti. Inoltre il fatto che i comandi siano oggetti distinti permette di poterli eventualmente aggregare insieme (applicando il pattern Composite) allo scopo di formare un comando complesso, costituito da una serie arbitraria di azioni.

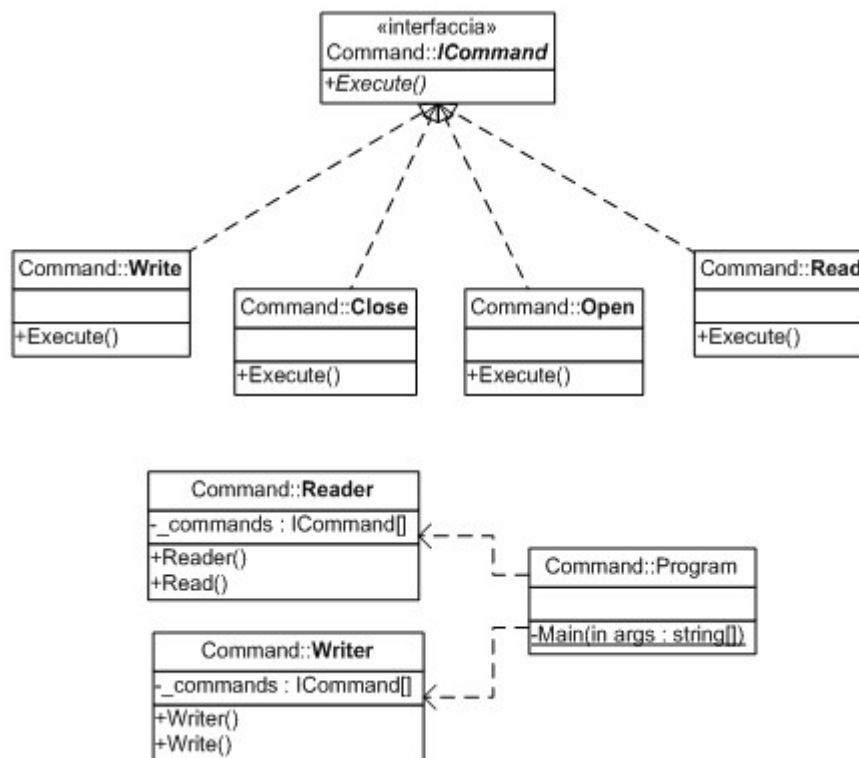


I partecipanti di questo pattern sono (tra parentesi sono indicati gli oggetti equivalenti nell'esempio proposto successivamente):

- **Command** (ICommand)
Definisce l'interfaccia di riferimento per ogni comando.
- **ConcreteCommand** (Open, Read, Write e Close)
Definisce un legame tra il **Receiver** e un'azione. Implementa in modo particolare il metodo `Execute()` invocando i metodi del **Receiver**.

- **Invoker** (Reader e Writer)
Aggrega i diversi comandi e delega a loro l'esecuzione delle azioni previste.
- **Receiver** (System.Console)
Conosce il modo di eseguire le operazioni associate ad una particolare richiesta.
- **Client** (Program)
Tramite l'Invoker attiva ed esegue un *ConcreteCommand* che va a interessare il *Receiver* corrispondente.

L'esempio proposto si riferisce all'esecuzione di due sequenze di comandi da parte delle classi Reader e Writer. L'esecuzione di ciascuna azione consiste nella scrittura sulla console di una stringa contenente il nome di ciascun comando appartenente alla sequenza. L'oggetto di destinazione pertanto è System.Console: Reader e Writer portano a compimento le operazioni desiderate sfruttando il suo metodo WriteLine(string). I singoli comandi sono classi diverse che implementano in modo particolare il metodo Execute(), definito nell'interfaccia ICommand. Si noti come il client (classe Program) sia completamente all'oscuro sia di come ciascun comando viene eseguito, sia di quale sequenza effettiva di azioni viene intrapresa di volta in volta.



```

using System;

namespace DesignPatterns.Command
{
    public interface ICommand
    {
        void Execute();
    }

    public class Open : ICommand
    {
        public virtual void Execute() { Console.WriteLine("Open"); }
    }

    public class Read : ICommand
    {
        public virtual void Execute() { Console.WriteLine("Read"); }
    }
}
  
```

```

public class Write : ICommand
{
    public virtual void Execute() { Console.WriteLine("Write"); }
}

public class Close : ICommand
{
    public virtual void Execute() { Console.WriteLine("Close"); }
}

public class Reader
{
    private ICommand[] _commands;

    public Reader()
    {
        _commands = new ICommand[] { new Open(), new Read(), new Close() };
    }
    public void Read()
    {
        foreach (ICommand cmd in _commands)
            cmd.Execute();
    }
}

public class Writer
{
    private ICommand[] _commands;

    public Writer()
    {
        _commands = new ICommand[] { new Open(), new Write(), new Close() };
    }

    public void Write()
    {
        foreach (ICommand cmd in _commands)
            cmd.Execute();
    }
}

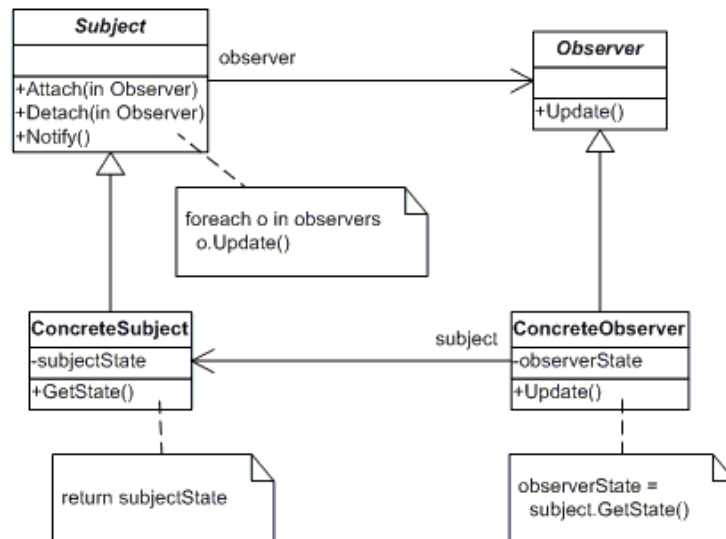
public class Program
{
    static void Main(string[] args)
    {
        Reader reader = new Reader();
        reader.Read();
        Writer writer = new Writer();
        writer.Write();
    }
}

```

Observer

Il pattern Observer (noto anche col nome Publish-Subscribe) permette di definire una dipendenza uno a molti fra oggetti, in modo tale che se un oggetto cambia il suo stato interno, ciascuno degli oggetti dipendenti da esso viene notificato e aggiornato automaticamente. L'Observer nasce dall'esigenza di mantenere un alto livello di consistenza fra classi correlate, senza peraltro produrre situazioni di forte dipendenza e accoppiamento elevato.

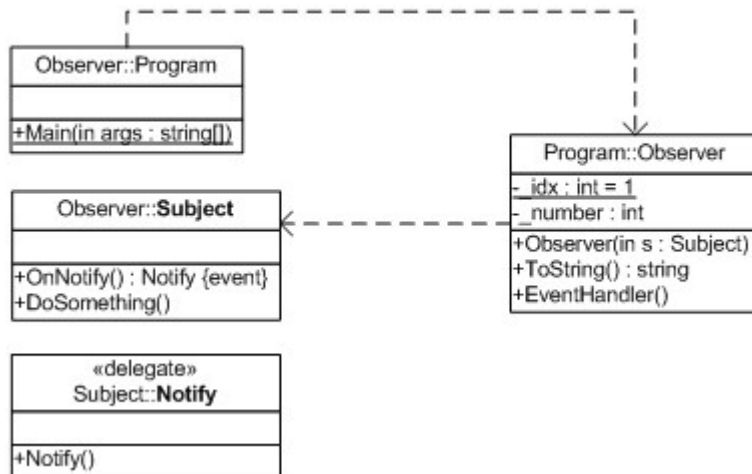
Il pattern Observer si presta ad essere utilizzato in diversi casi. Ad esempio, quando un'astrazione presenta due diversi aspetti tra loro dipendenti, è possibile definire due classi in cui incapsulare questi aspetti in modo tale da poterli utilizzare in maniera indipendente. In questo scenario occorre comunque prevedere un meccanismo di comunicazione che permetta di mantenere la consistenza tra le istanze delle due classi e il pattern Observer fornisce una soluzione elegante al problema senza generare accoppiamento. Un'altra situazione tipica di utilizzo si ha quando la modifica dello stato di un oggetto (per esempio, un controllo dell'interfaccia utente) implica un cambiamento dello stato di altri oggetti correlati, a prescindere dal loro numero (per esempio, altri controlli). In questo caso la modifica dello stato dell'oggetto (detto anche Publisher) si deve propagare agli oggetti correlati (detti anche Subscriber) in modo tale che essi possano aggiornare il loro stato interno di conseguenza.



I partecipanti di questo pattern sono (tra parentesi sono indicati gli oggetti equivalenti nell'esempio proposto successivamente):

- **Subject** (delegate Subject.Notify)
Conosce i suoi *Observer*. Fornisce l'interfaccia per associare e rimuovere oggetti *Observer*.
- **Observer**
Fornisce l'interfaccia di notifica per gli oggetti a cui devono essere segnalati i cambiamenti di stato di *Subject*.
- **ConcreteSubject** (Subject)
Contiene lo stato monitorato dagli *Observer* a cui viene inviata la notifica.
- **ConcreteObserver** (Observer)
Mantiene un riferimento ad un oggetto *ConcreteSubject*. Contiene le informazioni da mantenere sincronizzate con lo stato del *Subject*. Implementa il metodo di gestione della notifica da eseguire allo scopo di mantenere sincronizzati gli stati degli oggetti.

Il meccanismo per inviare notifiche nell'ambito del .NET Framework è fornito in modo nativo dai tipi delegate e dagli eventi. Una classe che funge da Publisher (classe Subject) espone in generale sulla sua interfaccia una serie di eventi corrispondenti ad un tipo particolare di delegate. Le classi Subscriber (classe Observer) sottoscrivono l'evento e ad esso associano un metodo interno (comunemente detto event handler) che deve rispettare la firma definita dal tipo delegate associato all'evento. L'event handler viene chiamato nel momento in cui il Publisher inoltra ai suoi Subscriber la notifica, rendendo possibile in questo modo l'esecuzione di codice in ciascun Subscriber al variare dello stato interno del Publisher.



```

using System;

namespace DesignPatterns.Observer
{
    public class Subject
    {
        public delegate void Notify();

        public event Notify OnNotify;

        public void DoSomething()
        {
            if (OnNotify != null)
            {
                Console.WriteLine("Subject fires event");
                OnNotify();
            }
        }
    }

    public class Program
    {
        public class Observer
        {
            private static int _idx = 1;
            private int _number;

            public Observer(Subject s)
            {
                s.OnNotify += new Subject.Notify(EventHandler);
                _number = _idx++;
            }

            public override string ToString()
            {
                return _number.ToString();
            }

            public void EventHandler()
            {
                Console.WriteLine("Observer {0} was called by subject", this);
            }
        }
    }
}

```

```

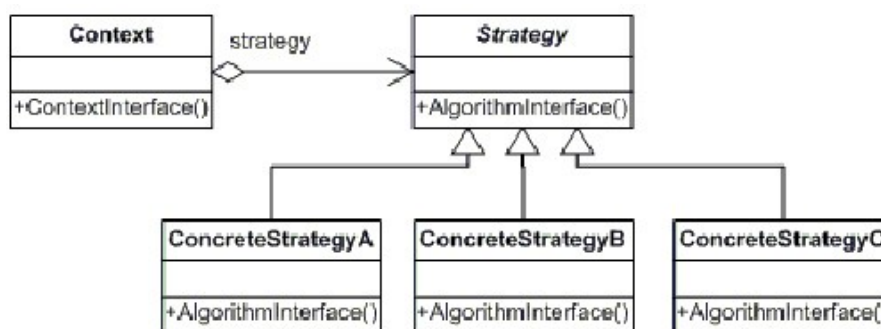
public static void Main(string[] args)
{
    Subject s = new Subject();
    Observer o1 = new Observer(s);
    Observer o2 = new Observer(s);
    s.DoSomething();
}
}
}

```

Strategy

Il pattern Strategy permette di definire una famiglia di algoritmi, di incapsularli e renderli intercambiabili fra loro. Questo pattern consente agli algoritmi di variare in modo indipendente rispetto al loro contesto di utilizzo, fornendo un basso accoppiamento tra le classi partecipanti del pattern e una alta coesione funzionale delle diverse strategie di implementazione.

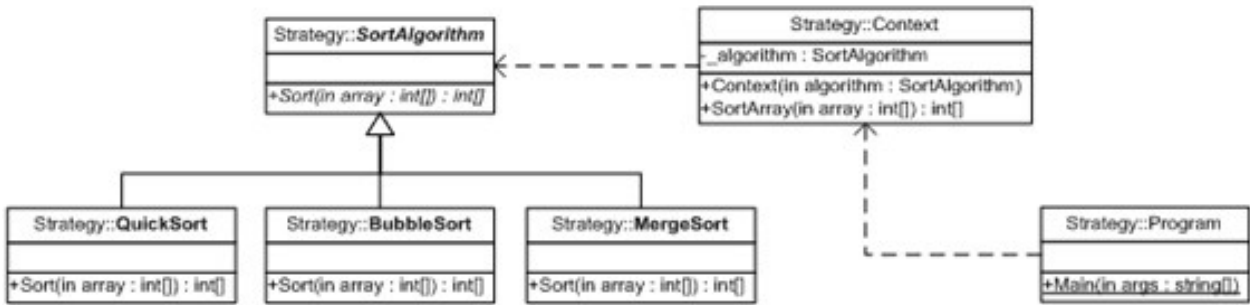
Il pattern Strategy fornisce di fatto un meccanismo di configurazione per una determinata classe, utilizzando un comportamento specifico scelto fra tanti e "iniettato" nel momento dell'utilizzo. L'operazione di iniezione può avvenire secondo diverse modalità, a seconda dei casi. La modalità più comune di iniezione prevede l'uso del costruttore della classe per selezionare l'algoritmo concreto da eseguire successivamente.



I partecipanti di questo pattern sono (tra parentesi sono indicati gli oggetti equivalenti nell'esempio proposto successivamente):

- **Strategy** (SortAlgorithm)
Dichiara l'interfaccia di riferimento per tutti gli algoritmi concreti.
- **ConcreteStrategy** (QuickSort, BubbleSort e MergeSort)
Implementa un particolare algoritmo utilizzando l'interfaccia definita da *Strategy*.
- **Context** (Context)
Carica un oggetto *ConcreteStrategy* e utilizza un riferimento a *Strategy* per eseguire l'algoritmo concreto. Definisce l'interfaccia per accedere ai membri dell'algoritmo caricato.

L'esempio proposto si riferisce all'utilizzo di un algoritmo per ordinare un insieme di numeri interi. Dal momento che l'ordinamento può essere fatto in svariati modi, esistono diverse versioni parallele dello stesso algoritmo. Incapsulando ciascuna di queste implementazioni in un oggetto specifico, è possibile renderle tra loro intercambiabili senza impattare sul codice che effettivamente utilizza l'algoritmo e sul risultato dell'operazione. La classe astratta SortAlgorithm rappresenta il tipo base da cui ogni implementazione dell'algoritmo deriva. Nell'esempio sono inclusi tre tipi distinti di ordinamento corrispondenti ad altrettante classi concrete: QuickSort, BubbleSort e MergeSort. La classe Context carica l'istanza di un algoritmo passata tramite il costruttore. Il metodo SortArray(int[]) di Context internamente utilizza il metodo Sort(int) di SortAlgorithm, implementato in modo particolare da ciascuna delle classi derivate. In questo modo il client (classe Program) utilizza unicamente Context per eseguire gli ordinamenti e qualsiasi modifica degli algoritmi e delle classi relative non impatta su di esso.



```

using System;

namespace DesignPatterns.Strategy
{
    public abstract class SortAlgorithm
    {
        public abstract int[] Sort(int[] array);
    }

    public class QuickSort : SortAlgorithm
    {
        public override int[] Sort(int[] array)
        {
            Array.Sort<int>(array);
            return array;
        }
    }

    public class BubbleSort : SortAlgorithm
    {
        public override int[] Sort(int[] array)
        { throw new NotImplementedException(); }
    }

    public class MergeSort : SortAlgorithm
    {
        public override int[] Sort(int[] array)
        { throw new NotImplementedException(); }
    }

    public class Context
    {
        private SortAlgorithm _algorithm;

        public Context(SortAlgorithm algorithm)
        {
            _algorithm = algorithm;
        }

        public int[] SortArray(int[] array)
        {
            return _algorithm.Sort(array);
        }
    }

    public class Program
    {
        public static void Main(string[] args)
        {
            int[] array = { 21, 10, 71, 18, 8, 5, 20, 1, 67 };
        }
    }
}

```

```

Context ctx = new Context(new QuickSort());
array = ctx.SortArray(array);

foreach (int i in array)
    Console.WriteLine(i.ToString());

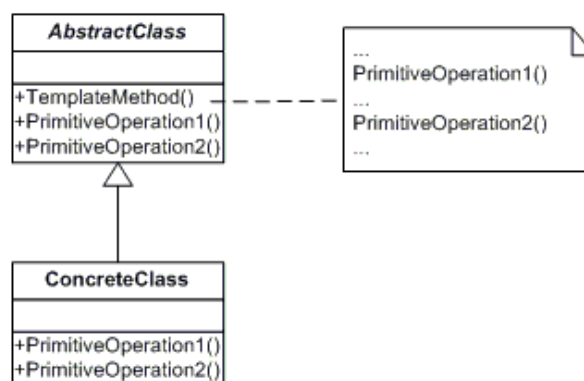
Console.ReadLine();
    }
}

```

Template Method

Il pattern Template Method permette di definire la struttura di un algoritmo all'interno di un metodo di una superclasse (detto appunto metodo template), delegando alcuni passi alle classi derivate. Questo pattern lascia che le classi derivate possano definire in modo particolare alcuni passi dell'algoritmo senza dover implementare ogni volta da zero la struttura dell'algoritmo stesso.

Il pattern Template Method fornisce un meccanismo molto efficace di riuso del codice. I metodi template infatti richiamano per lo più una serie di metodi astratti che devono essere implementati in modo particolare nelle classi derivate. Questo fa sì che nelle classi derivate non debba essere fornita ogni volta l'implementazione di tutto l'algoritmo, ma solamente delle parti salienti, limitando in questo modo le duplicazioni e migliorando la leggibilità e la manutenibilità del codice. Talvolta, invece dei metodi astratti, nella superclasse può risultare comodo utilizzare metodi virtuali per rappresentare i diversi passi dell'algoritmo. In questo caso la classe base fornisce per ciascun passo dell'algoritmo un'implementazione predefinita e la personalizzazione nelle classi derivate può essere omessa qualora non risulti necessaria.

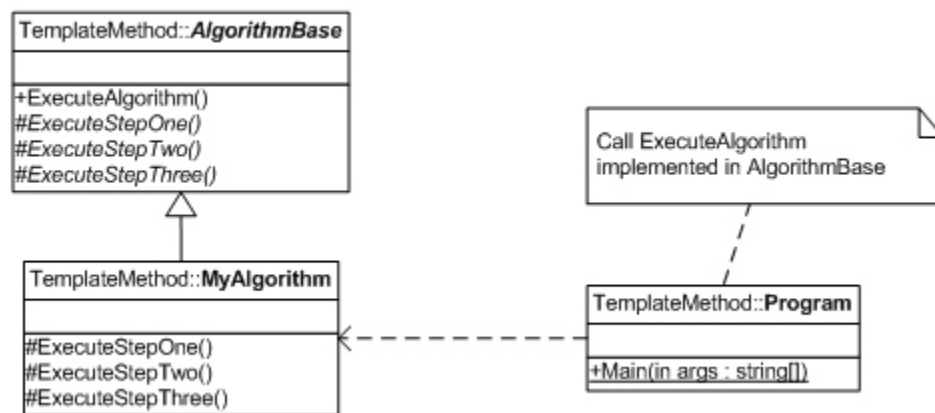


I partecipanti di questo pattern sono (tra parentesi sono indicati gli oggetti equivalenti nell'esempio proposto successivamente):

- **AbstractClass** (AlgorithmBase)
Definisce i vari metodi astratti che rappresentano i diversi passi di un determinato algoritmo. Include un metodo generale che definisce la struttura dell'algoritmo e ingloba le chiamate ai vari metodi astratti.
- **ConcreteClass** (MyAlgorithm)
Fornisce un'implementazione concreta dei vari metodi astratti definiti in *AbstractClass*.

Nell'esempio proposto la funzione `ExecuteAlgorithm()` della classe `AlgorithmBase` è un metodo template. Internamente esso richiama tre metodi protetti e astratti corrispondenti ai vari passi dell'algoritmo da eseguire. Dal momento che `AlgorithmBase` è una classe astratta, i suoi membri astratti devono essere obbligatoriamente implementati nelle classi derivate (classe `MyAlgorithm`). Peraltro l'implementazione presente in `MyAlgorithm` non va ad interessare l'interfaccia pubblica definita nel tipo base, dal momento che

i vari metodi astratti sono visibili unicamente nelle classi derivate in quanto protetti. In questo modo è possibile variare l'algoritmo per ogni diversa specializzazione di AlgorithmBase semplicemente ridefinendo di volta in volta i vari passi dell'algoritmo, senza peraltro modificare la struttura del tipo base e il modo in cui esso viene utilizzato nell'ambito del codice del client (classe Program).



```

using System;

namespace DesignPatterns.TemplateMethod
{
    public abstract class AlgorithmBase
    {
        public void ExecuteAlgorithm()
        {
            ExecuteStepOne();
            ExecuteStepTwo();
            ExecuteStepThree();
        }

        protected abstract void ExecuteStepOne();
        protected abstract void ExecuteStepTwo();
        protected abstract void ExecuteStepThree();
    }

    public class MyAlgorithm : AlgorithmBase
    {
        protected override void ExecuteStepOne() { Console.WriteLine("Step 1"); }

        protected override void ExecuteStepTwo() { Console.WriteLine("Step 2"); }

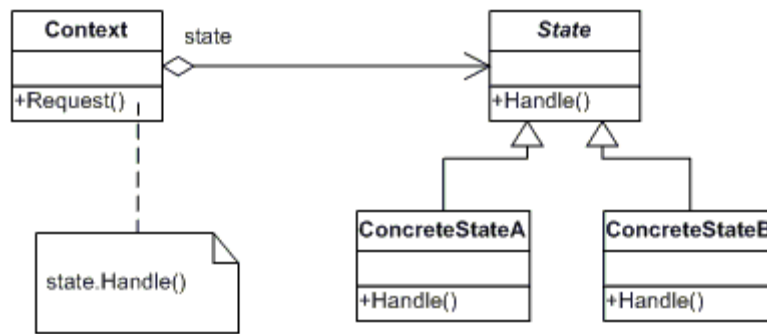
        protected override void ExecuteStepThree() { Console.WriteLine("Step 3"); }
    }

    public class Program
    {
        public static void Main(string[] args)
        {
            new MyAlgorithm().ExecuteAlgorithm();
        }
    }
}

```

State

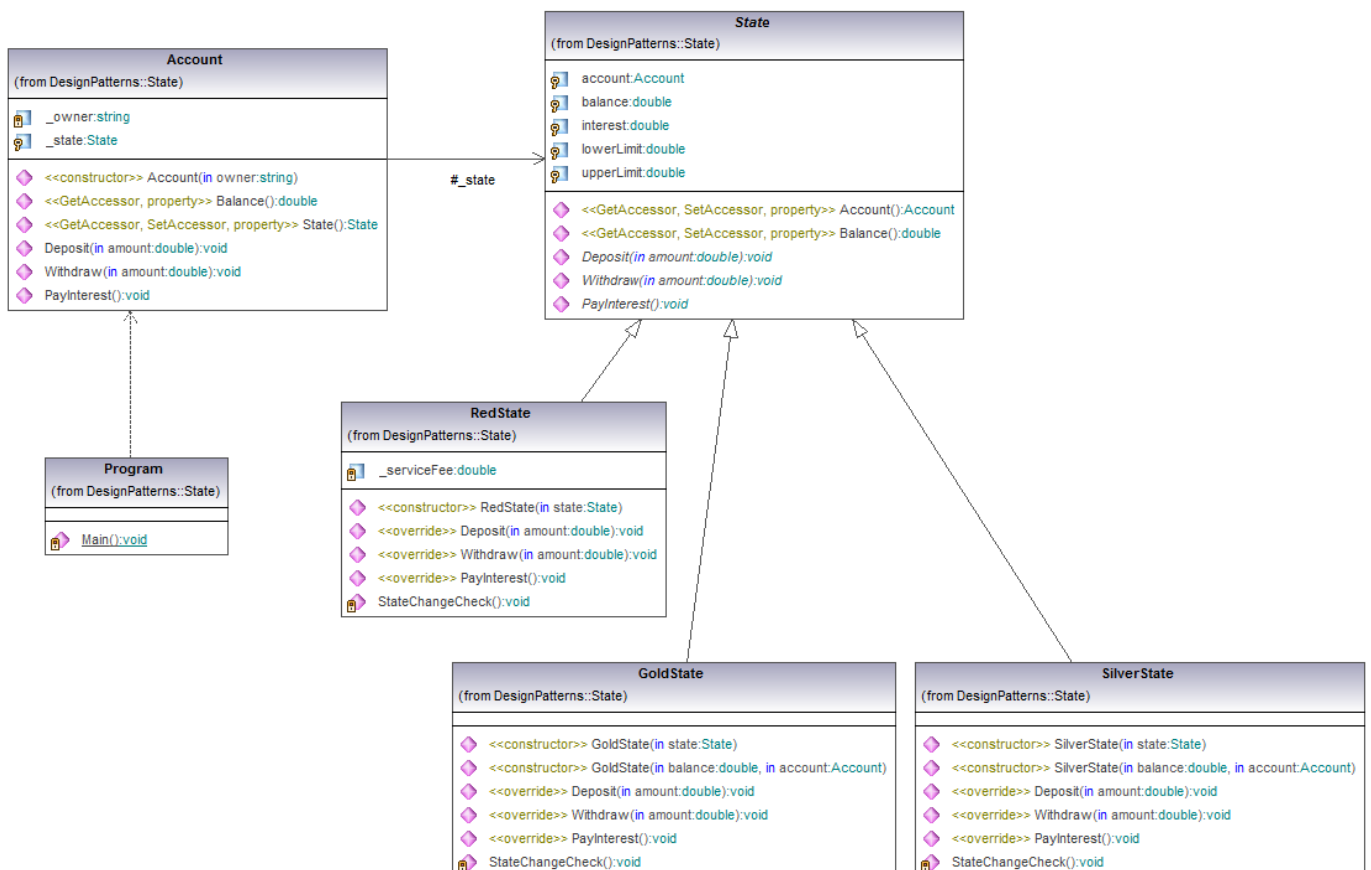
Esso consente ad un oggetto di cambiare il proprio comportamento a run-time in funzione dello stato in cui si trova.



I partecipanti di questo pattern sono (tra parentesi sono indicati gli oggetti equivalenti nell'esempio proposto successivamente):

- **Context** (Account)
 - Definisce l'interfaccia del client e mantiene un riferimento ad un ConcreteState.
- **State** (State)
 - Definisce l'interfaccia, implementata dai ConcreteState, che incapsula la logica del comportamento associato ad un determinato stato.
- **ConcreteState** (RedState, SilverState, GoldState)
 - Implementa il comportamento associato ad un particolare stato.

Il comportamento associato ad uno stato dipende solo da una classe (ConcreteState). La logica che implementa il cambiamento di stato viene implementata in una sola classe (Context) piuttosto che con istruzioni condizionali (if o switch) nella classe che implementa il comportamento. Il seguente codice mostra il pattern State il quale permette ad un oggetto di comportarsi diversamente in base ad un proprio stato. La differenza nel comportamento è delegata agli oggetti che rappresentano questo stato.



```

using System;

namespace DesignPatterns.State
{
    abstract class State
    {
        protected Account account;
        protected double balance;
        protected double interest;
        protected double lowerLimit;
        protected double upperLimit;

        public Account Account
        {
            get { return account; }
            set { account = value; }
        }

        public double Balance
        {
            get { return balance; }
            set { balance = value; }
        }

        public abstract void Deposit(double amount);
        public abstract void Withdraw(double amount);
        public abstract void PayInterest();
    }

    class RedState : State
    {
        private double _serviceFee;

        public RedState(State state)
        {
            this.balance = state.Balance;
            this.account = state.Account;
            interest = 0.0; lowerLimit = -100.0; upperLimit = 0.0;
            _serviceFee = 15.00;
        }

        public override void Deposit(double amount)
        {
            balance += amount;
            StateChangeCheck();
        }

        public override void Withdraw(double amount)
        {
            amount = amount - _serviceFee;
            Console.WriteLine("No funds available for withdrawal!");
        }

        public override void PayInterest() { }

        private void StateChangeCheck()
        {
            if (balance > upperLimit)
            {
                account.State = new SilverState(this);
            }
        }
    }
}

```

```

class SilverState : State
{
    public SilverState(State state) :
        this(state.Balance, state.Account)
    {
    }

    public SilverState(double balance, Account account)
    {
        this.balance = balance;
        this.account = account;
        interest = 0.0; lowerLimit = 0.0; upperLimit = 1000.0;
    }

    public override void Deposit(double amount)
    {
        balance += amount;
        StateChangeCheck();
    }

    public override void Withdraw(double amount)
    {
        balance -= amount;
        StateChangeCheck();
    }

    public override void PayInterest()
    {
        balance += interest * balance;
        StateChangeCheck();
    }

    private void StateChangeCheck()
    {
        if (balance < lowerLimit)
        {
            account.State = new RedState(this);
        }
        else if (balance > upperLimit)
        {
            account.State = new GoldState(this);
        }
    }
}

class GoldState : State
{
    public GoldState(State state)
        : this(state.Balance, state.Account)
    {
    }

    public GoldState(double balance, Account account)
    {
        this.balance = balance;
        this.account = account;
        interest = 0.05; lowerLimit = 1000.0; upperLimit = 10000000.0;
    }

    private void StateChangeCheck()
    {
        if (balance < 0.0)
        {

```

```

        account.State = new RedState(this);
    }
    else if (balance < lowerLimit)
    {
        account.State = new SilverState(this);
    }
}

public override void Deposit(double amount)
{
    balance += amount;
    StateChangeCheck();
}

public override void Withdraw(double amount)
{
    balance -= amount;
    StateChangeCheck();
}

public override void PayInterest()
{
    balance += interest * balance;
    StateChangeCheck();
}
}

class Account
{
    private State _state;
    private string _owner;

    public Account(string owner)
    {
        this._owner = owner;
        this._state = new SilverState(0.0, this);
    }

    public double Balance
    {
        get { return _state.Balance; }
    }

    public State State
    {
        get { return _state; }
        set { _state = value; }
    }

    public void Deposit(double amount)
    {
        _state.Deposit(amount);
        Console.WriteLine("Deposited {0:C} --- ", amount);
        Console.WriteLine(" Balance = {0:C}", this.Balance);
        Console.WriteLine(" Status = {0}", this.State.GetType().Name);
    }

    public void Withdraw(double amount)
    {
        _state.Withdraw(amount);
        Console.WriteLine("Withdrew {0:C} --- ", amount);
        Console.WriteLine(" Balance = {0:C}", this.Balance);
        Console.WriteLine(" Status = {0}\n", this.State.GetType().Name);
    }
}

```

```

    public void PayInterest()
    {
        _state.PayInterest();
        Console.WriteLine("Interest Paid --- ");
        Console.WriteLine(" Balance = {0:C}", this.Balance);
        Console.WriteLine(" Status = {0}\n", this.State.GetType().Name);
    }
}

class Program
{
    static void Main()
    {
        Account account = new Account("Jim Johnson");

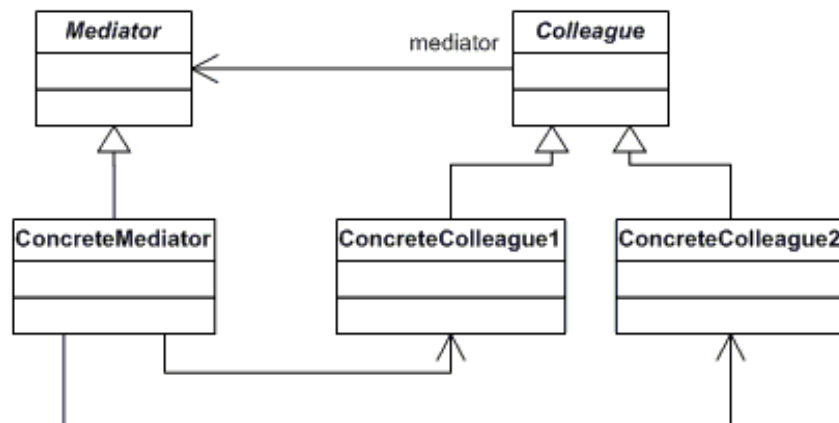
        account.Deposit(500.0);
        account.Deposit(300.0);
        account.Deposit(550.0);
        account.PayInterest();
        account.Withdraw(2000.00);
        account.Withdraw(1100.00);
    }
}

```

Mediator

Si tratta di un pattern comportamentale, ossia operante nel contesto delle interazioni tra oggetti, che ha l'intento di disaccoppiare entità del sistema che devono comunicare fra loro. Il pattern infatti fa in modo che queste entità non si riferiscano reciprocamente, agendo da "mediatore" fra le parti.

Il beneficio principale è il permettere di modificare agilmente le politiche di interazione, poiché le entità coinvolte devono fare riferimento al loro interno solamente al mediatore.



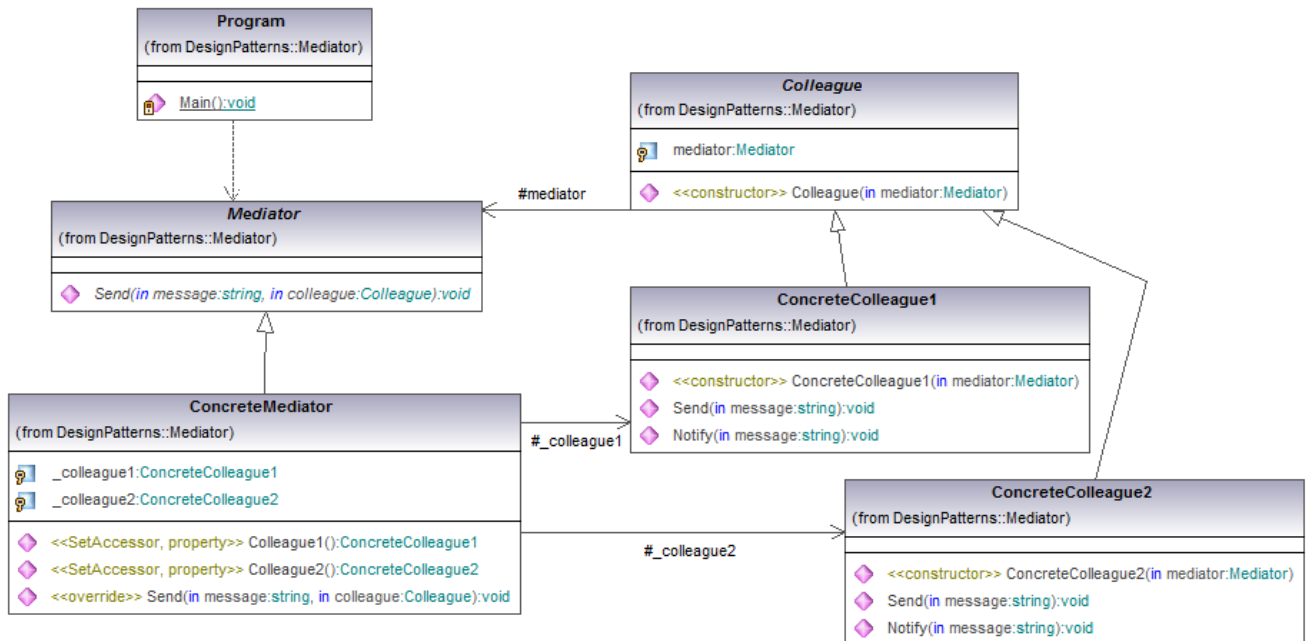
I partecipanti di questo pattern sono (tra parentesi sono indicati gli oggetti equivalenti nell'esempio proposto successivamente):

- **Mediator** (IChatroom)
 - Funge da intermediario fra i vari colleagues, oggetti dei quali mantiene una lista interna. Riceve gli eventi da essi e modifica lo stato del sistema di conseguenza.
- **ConcreteMediator** (Chatroom)
 - Implementa il comportamento cooperativo coordinando gli oggetti Colleague. Inoltre, conosce e conserva i propri Colleague.

- **Colleague classes** (Participant)

- Ogni classe Colleague conosce il proprio oggetto Mediator. Inoltre, invia notifiche al mediator quando il suo stato cambia: cioè quando comunica con un altro collega.

Il seguente codice mostra l'uso del pattern Mediator che facilita il basso accoppiamento nella comunicazione tra i diversi partecipanti registrati nella Chatroom. La Chatroom è il nodo centrale attraverso il quale avvengono tutte le comunicazioni. A questo punto solo una comunicazione uno-a-uno è implementata nella Chatroom, ma è banale trasformarla in una comunicazione una-a-molti.



```

using System;

namespace DesignPatterns.Mediator
{
    abstract class Mediator
    {
        public abstract void Send(string message, Colleague colleague);
    }

    class ConcreteMediator : Mediator
    {
        private ConcreteColleague1 _colleague1;
        private ConcreteColleague2 _colleague2;

        public ConcreteColleague1 Colleague1
        {
            set { _colleague1 = value; }
        }

        public ConcreteColleague2 Colleague2
        {
            set { _colleague2 = value; }
        }

        public override void Send(string message, Colleague colleague)
        {
            if (colleague == _colleague1)
            {
                _colleague2.Notify(message);
            }
        }
    }
}

```

```

        else
        {
            _colleague1.Notify(message);
        }
    }
}

abstract class Colleague
{
    protected Mediator mediator;

    public Colleague(Mediator mediator)
    {
        this.mediator = mediator;
    }
}

class ConcreteColleague1 : Colleague
{
    public ConcreteColleague1(Mediator mediator)
        : base(mediator)
    {
    }

    public void Send(string message)
    {
        mediator.Send(message, this);
    }

    public void Notify(string message)
    {
        Console.WriteLine("Colleague1 gets message: " + message);
    }
}

class ConcreteColleague2 : Colleague
{
    public ConcreteColleague2(Mediator mediator)
        : base(mediator)
    {
    }

    public void Send(string message)
    {
        mediator.Send(message, this);
    }

    public void Notify(string message)
    {
        Console.WriteLine("Colleague2 gets message: " + message);
    }
}

class Program
{
    static void Main()
    {
        ConcreteMediator m = new ConcreteMediator();
        ConcreteColleague1 c1 = new ConcreteColleague1(m);
        ConcreteColleague2 c2 = new ConcreteColleague2(m);
        m.Colleague1 = c1;
        m.Colleague2 = c2;
    }
}

```

```

        c1.Send("How are you?");
        c2.Send("Fine, thanks");
    }
}
}

```

I design pattern creazionali

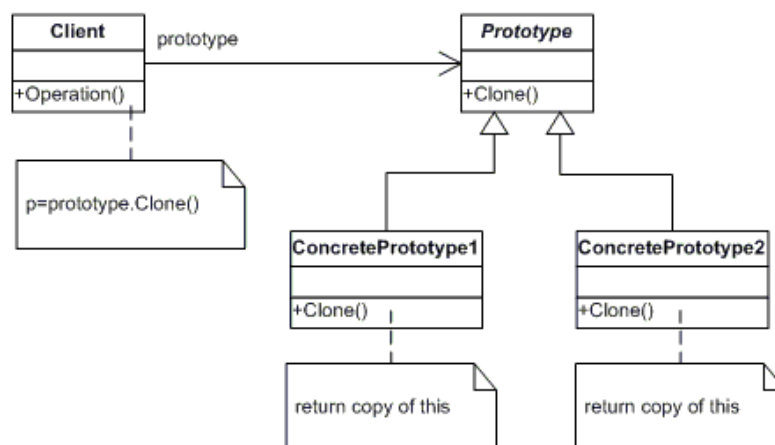
I pattern creazionali forniscono un'astrazione del processo di creazione delle istanze delle classi, favorendo l'indipendenza del sistema dalle modalità di creazione e dai tipi concreti effettivamente generati. Ci sono due aspetti che caratterizzano i pattern appartenenti a questa categoria:

- la capacità di mascherare al client la conoscenza degli oggetti concreti creati, sfruttando tipi astratti per definire le interfacce di riferimento;
- la capacità di nascondere le modalità di creazione all'utilizzatore dell'istanza.

Queste due caratteristiche conferiscono una notevole flessibilità al processo di creazione, dal momento che ciò che viene creato in generale risulta essere disaccoppiato dal contesto di utilizzo. Infatti solo l'oggetto creatore conosce il tipo effettivo dell'istanza e ciò che viene esternamente reso pubblico è unicamente l'interfaccia di riferimento.

Prototype

Prototype è un design pattern creazionale utilizzato in informatica nella programmazione orientata agli oggetti. Prototype permette di creare nuovi oggetti clonando un oggetto iniziale, detto appunto prototipo. A differenza di altri pattern come Abstract factory o Factory method permette di specificare nuovi oggetti a tempo d'esecuzione (run-time), utilizzando un gestore di prototipi (prototype manager) per salvare e reperire dinamicamente le istanze degli oggetti desiderati.



I partecipanti di questo pattern sono (tra parentesi sono indicati gli oggetti equivalenti nell'esempio proposto successivamente):

- **Prototype** (ColorPrototype)
Definisce l'interfaccia da utilizzare per poterlo clonare.
- **ConcretePrototype** (Color)
Implementa l'operazione per clonarsi.
- **Client** (Program)
Crea un nuovo oggetto chiedendo al prototipo di clonare se stesso.

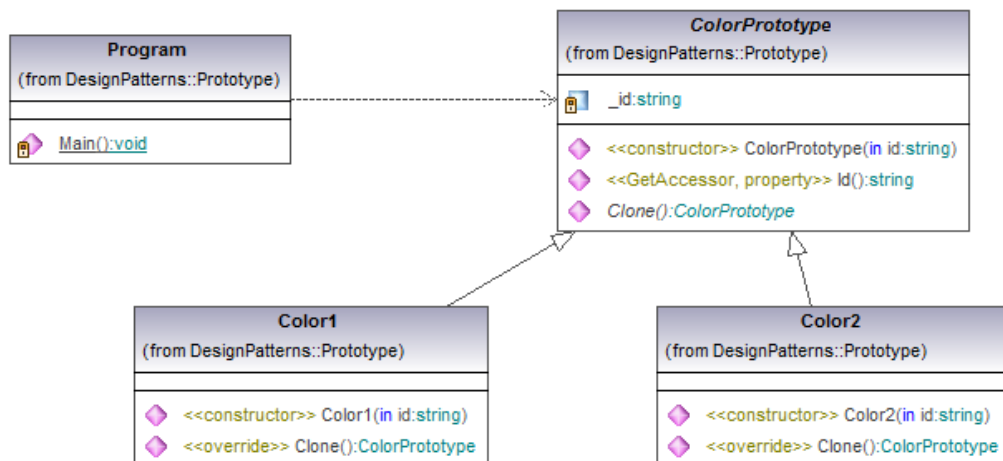
Come altri pattern creazionali, ovvero che si occupano di istanziare oggetti, prototype mira a rendere indipendente un sistema dal modo in cui i suoi oggetti vengono creati. Inoltre può rivelarsi utile quando:

- le classi da istanziare sono specificate solamente a tempo d'esecuzione, per cui un codice statico non può occuparsi della creazione dell'oggetto, oppure
- per evitare di costruire una gerarchia di factory in parallelo a una gerarchia di prodotti, come avviene utilizzando Abstract factory e Factory method, oppure
- quando le istanze di una classe possono avere soltanto un limitato numero di stati, per cui può essere più conveniente clonare al bisogno il prototipo corrispondente piuttosto che creare l'oggetto e configurarlo ogni volta.

Prototype è più flessibile di altri pattern creazionali, perché l'aggiunta di un prodotto richiede semplicemente la registrazione dell'oggetto da clonare in un gestore di prototipi (prototype manager), descritto nella sezione implementazione. Questa caratteristica permette a un Client di aggiungere un prodotto a tempo d'esecuzione, e di renderlo disponibile per la clonazione ad altri Client.

Quando si devono definire numerosi oggetti differenziati tra loro solo dai valori che assumono le loro variabili interne è più comodo istanziare nuovi oggetti semplicemente clonando un prototipo iniziale e successivamente impostare la rappresentazione interna perché assuma la configurazione desiderata. Per esempio un editor di spartiti musicali potrebbe istanziare un solo prototipo di nota, clonarlo e impostare altezza e durata invece di definire una classe per ogni nota.

Nelle applicazioni che aggregano oggetti diversi in strutture composte, magari utilizzando i pattern Composite o Decorator, l'utilizzo di prototipi può semplificare la gestione e la generazione di parti e sottoparti. Per esempio, un programma di grafica vettoriale potrebbe permettere all'utente di salvare composizioni di oggetti per poi poterne generare copie al bisogno.



```

using System;

namespace DesignPatterns.Prototype
{
    abstract class ColorPrototype
    {
        private string _id;

        public ColorPrototype(string id)
        {
            this._id = id;
        }

        public string Id
  
```

```

    {
        get { return _id; }
    }

    public abstract ColorPrototype Clone();
}

class Color1 : ColorPrototype
{
    public Color1(string id)
        : base(id)
    {
    }

    public override ColorPrototype Clone()
    {
        return (ColorPrototype) this.MemberwiseClone();
    }
}

class Color2 : ColorPrototype
{
    public Color2(string id)
        : base(id)
    {
    }

    public override ColorPrototype Clone()
    {
        return (ColorPrototype) this.MemberwiseClone();
    }
}

class Program
{
    static void Main()
    {
        Color1 p1 = new Color1("I");
        Color1 c1 = (Color1)p1.Clone();
        Console.WriteLine("Cloned: {0}", c1.Id);

        Color2 p2 = new ConcretePrototype2("II");
        Color2 c2 = (Color2)p2.Clone();
        Console.WriteLine("Cloned: {0}", c2.Id);
    }
}

```

Conclusioni

L'implementazione di un pattern non deve necessariamente seguire lo schema di base. La versione, per così dire, ufficiale di un pattern deve rappresentare semplicemente un punto da cui partire per operare le scelte implementative utili a risolvere un determinato problema di disegno e di sviluppo. Sta a chi progetta e scrive il codice trovare di volta in volta le soluzioni giuste, spesso dettate non solo dalla conoscenza della teoria, ma anche e soprattutto dal buon senso e dall'esperienza personale.